```
"use client";

import { useEffect, useState, useRef } from "react";
import { MoreVertical, FolderOpen, CheckCircle2 } from "lucide-react";
import { useToast, ToastContainer } from "@/components/Toast";
import { ConfirmDialog } from "@/components/ConfirmDialog";
import DescriptionDisplay from "@/components/DescriptionDisplay";
import UrlVideoPopup from "@/components/UrlVideoPopup";
import UpdateAttachmentDialog from
"@/components/UpdateAttachmentDialog";

type Option = { id: number; name: string };

type ProgramOption = {
  id: number;
  name: string;
  client?: number;
};

type Assignment = {
  id: number;
  client: Option;
  program: ProgramOption;
  created_at?: string;
};
```

```
type Attachment = {
  id: number;
  file_name: string;
  description: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
  clients: Option[]; // temporary legacy support
  programs: ProgramOption[]; // temporary legacy support
  assignments: Assignment[]; // NEW and important
};

export default function UploadedFilesList({
  refreshKey,
  clientId,
  programId,
}: {
  refreshKey?: number;
  clientId?: number | null;
  programId?: number | null;
}) {
  const API = process.env.NEXT_PUBLIC_API_BASE_URL;

  const [files, setFiles] = useState<Attachment[]>([]);
  const [openMenu, setOpenMenu] = useState<number | null>(null);
  const [search, setSearch] = useState("");
  const [fileIdToDelete, setFileIdToDelete] = useState<number | null>
(null);
  const [videoPopup, setVideoPopup] = useState<{
```

```
    title: string;
    videoUrl: string;
    embedUrl: string | null;
    isDirectVideo: boolean;
  } | null>(null);
  const [fileToUpdate, setFileToUpdate] = useState<Attachment | null>
(null);
  const [updateFile, setUpdateFile] = useState<File | null>(null);
  const [updateUrl, setUpdateUrl] = useState("");
  const [updateLoading, setUpdateLoading] = useState(false);

  const { toasts, showToast, removeToast } = useToast();
    /** 1GB limit */
  const MAX_FILE_SIZE = 1024 * 1024 * 1024;

  /** Allowed MIME prefixes & types */
  const ALLOWED_TYPES = [
    "image/",
    "application/pdf",
    "application/msword",
    "application/vnd.ms-excel",
    "application/vnd.ms-powerpoint",
    "application/vnd.openxmlformats-officedocument",
    "text/plain",
    "video/",
    "audio/",
  ];

  function validateFile(file: File) {
```

```
    if (file.size > MAX_FILE_SIZE) {
      return "File size exceeds 1GB limit";
    }

    const isAllowed = ALLOWED_TYPES.some((type) =>
      file.type.startsWith(type)
    );

    if (!isAllowed) {
      return "Unsupported file type";
    }

    return null;
  }

  function openUpdateModal(file: Attachment) {
    setOpenMenu(null);
    setFileToUpdate(file);
    setUpdateFile(null);
    setUpdateUrl(file.external_url || "");
  }
  function notifyAttachmentsUpdated() {
    localStorage.setItem("attachments_updated_at",
Date.now().toString());
    window.dispatchEvent(new Event("attachments-updated"));
  }

  function normalizeAttachment(raw: any): Attachment {
    return {
```

```
    id: Number(raw?.id),
    file_name: raw?.file_name || "",
    description: raw?.description || "",
    asset_type: raw?.asset_type || "file",
    sharepoint_url: raw?.sharepoint_url || null,
    external_url: raw?.external_url || null,
    clients: Array.isArray(raw?.clients) ? raw.clients : [],
    programs: Array.isArray(raw?.programs) ? raw.programs : [],
    assignments: Array.isArray(raw?.assignments) ? raw.assignments : [],
  };
}

async function fetchFilesREST() {
  try {
    const res = await fetch(`${API}/api/attachments/`, {
      cache: "no-store",
    });

    const data = await res.json().catch(() => []);

    if (!res.ok) {
      throw new Error(data?.error || "Failed to fetch attachments");
    }

    const normalized = Array.isArray(data)
      ? data.map(normalizeAttachment)
      : [];

    // TEMP DEBUG - remove later if you want
```

```
      console.log("ATTACHMENTS API DATA:", normalized);

      setFiles(normalized);
    } catch (err: any) {
      console.error("Failed to fetch attachments:", err);
      showToast(err.message || "Failed to load files", "error");
      setFiles([]);
    }
  }

  useEffect(() => {
    fetchFilesREST();
  }, [refreshKey]);

  // IMPORTANT: listen for global updates from other components
  useEffect(() => {
    function handleAttachmentsUpdated() {
      fetchFilesREST();
    }

    window.addEventListener("attachments-updated",
handleAttachmentsUpdated);
    return () => {
      window.removeEventListener("attachments-updated",
handleAttachmentsUpdated);
    };
  }, []);

  async function assignFileToCurrentSelection(file: Attachment) {
```

```
    if (!clientId || !programId) {
      showToast("Please select a Client and Program from the sidebar
first", "error");
      return;
    }

    try {
      const res = await
fetch(`${API}/api/attachments/${file.id}/assign/`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          client_id: Number(clientId),
          program_id: Number(programId),
        }),
      });

      const data = await res.json().catch(() => ({}));

      if (!res.ok) {
        throw new Error(
          data?.program_id?.[0] ||
            data?.non_field_errors?.[0] ||
            data?.error ||
            "Failed to assign file"
        );
      }

      const updatedAttachment = data?.attachment
```

```
      ? normalizeAttachment(data.attachment)
      : null;

    // Update local state immediately for instant UI response
    if (updatedAttachment) {
      setFiles((prev) =>
        prev.map((f) => (f.id === file.id ? updatedAttachment : f))
      );
    }

    setOpenMenu(null);

    showToast(
      data?.created === false
        ? "File already assigned to selected client/program"
        : "File assigned successfully",
      "success"
    );

    notifyAttachmentsUpdated();

    // SAFEST: force re-fetch so assigned section is always correct
    await fetchFilesREST();
  } catch (err: any) {
    showToast(err.message || "Assignment failed", "error");
  }
}

async function unassignFile(file: Attachment) {
```

```
    if (!clientId || !programId) {
      showToast("Please select a Client and Program first", "error");
      return;
    }

    try {
      const res = await
fetch(`${API}/api/attachments/${file.id}/unassign/`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          client_id: Number(clientId),
          program_id: Number(programId),
        }),
      });

      const data = await res.json().catch(() => ({}));

      if (!res.ok) {
        throw new Error(
          data?.program_id?.[0] ||
            data?.non_field_errors?.[0] ||
            data?.error ||
            "Failed to unassign file"
        );
      }

      const updatedAttachment = data?.attachment
        ? normalizeAttachment(data.attachment)
```

```
      : null;

    // Update local state immediately for instant UI response
    if (updatedAttachment) {
      setFiles((prev) =>
        prev.map((f) => (f.id === file.id ? updatedAttachment : f))
      );
    }

    setOpenMenu(null);

    showToast(
      data?.deleted
        ? "File unassigned successfully"
        : "File was already unassigned",
      "success"
    );

    notifyAttachmentsUpdated();

    // SAFEST: force re-fetch so assigned section is always correct
    await fetchFilesREST();
  } catch (err: any) {
    showToast(err.message || "Unassign failed", "error");
  }
}
async function confirmUpdateFile() {
  if (!fileToUpdate) return;
```

```
    if (!updateFile && !updateUrl.trim()) {
      showToast("Please select a new file or provide a new URL", "error");
      return;
    }

    // Prevent ambiguity
    if (updateFile && updateUrl.trim()) {
      showToast("Please provide either a new file or a new URL, not
both", "error");
      return;
    }

    if (updateFile) {
      const validationError = validateFile(updateFile);
      if (validationError) {
        showToast(validationError, "error");
        return;
      }
    }

    setUpdateLoading(true);

    try {
      const fd = new FormData();

      if (updateFile) {
        fd.append("file", updateFile);
      }
```

```
if (updateUrl.trim()) {
  fd.append("external_url", updateUrl.trim());
}

// Keep same existing values
fd.append("name", fileToUpdate.file_name);
fd.append("description", fileToUpdate.description || "");

const res = await fetch(
  `${API}/api/attachments/${fileToUpdate.id}/replace/`,
  {
    method: "POST",
    body: fd,
    credentials: "include",
  }
);

const data = await res.json().catch(() => ({}));

if (!res.ok) {
  throw new Error(data?.error || "Failed to update file");
}

const updatedAttachment = data?.attachment
  ? normalizeAttachment(data.attachment)
  : null;

// instant UI refresh
if (updatedAttachment) {
```

```
        setFiles((prev) =>
          prev.map((f) => (f.id === updatedAttachment.id ?
updatedAttachment : f))
        );
      }

      // close modal + reset state
      setFileToUpdate(null);
      setUpdateFile(null);
      setUpdateUrl("");

      showToast("File updated successfully", "success");

      notifyAttachmentsUpdated();

      // safest final refresh
      await fetchFilesREST();
    } catch (err: any) {
      showToast(err.message || "Update failed", "error");
    } finally {
      setUpdateLoading(false);
    }
  }

  function requestDeleteFile(fileId: number) {
    setOpenMenu(null);
    setFileIdToDelete(fileId);
  }
```

```
async function confirmDeleteFile() {
  if (fileIdToDelete == null) return;

  const fileId = fileIdToDelete;
  setFileIdToDelete(null);

  try {
    const res = await fetch(`${API}/api/attachments/${fileId}/`, {
      method: "DELETE",
    });

    const data = await res.json().catch(() => ({}));

    if (!res.ok) {
      throw new Error(data?.error || "Failed to delete file");
    }

    setFiles((prev) => prev.filter((f) => f.id !== fileId));
    showToast("File deleted successfully", "success");
    notifyAttachmentsUpdated();

    // optional safety refetch
    await fetchFilesREST();
  } catch (err: any) {
    showToast(err.message || "Delete failed", "error");
  }
}

function isDirectVideoUrl(url: string) {
```

```
  const lower = url.toLowerCase();
  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v")
  );
}

function getEmbedUrl(url: string): string | null {
  try {
    const parsed = new URL(url);
    const host = parsed.hostname.toLowerCase();

    // YouTube short link: youtu.be/VIDEO_ID
    if (host.includes("youtu.be")) {
      const videoId = parsed.pathname.replace("/", "");
      return videoId ? `https://www.youtube.com/embed/${videoId}` :
null;
    }

    // YouTube normal link: youtube.com/watch?v=VIDEO_ID
    if (host.includes("youtube.com")) {
      const videoId = parsed.searchParams.get("v");
      if (videoId) return `https://www.youtube.com/embed/${videoId}`;

      // already embed url
      if (parsed.pathname.includes("/embed/")) return url;
```

```
    }

    // Vimeo
    if (host.includes("vimeo.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);
      const videoId = pathParts[pathParts.length - 1];
      return videoId ? https://player.vimeo.com/video/${videoId} :
null;
    }

    // Loom
    if (host.includes("loom.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);

      const shareIndex = pathParts.findIndex((part) => part ===
"share");
      if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
        return https://www.loom.com/embed/${pathParts[shareIndex +
1]};
      }

      const embedIndex = pathParts.findIndex((part) => part ===
"embed");
      if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
        return url;
      }
    }

    return null;
```

```
  } catch {
    return null;
  }
}

function isPlayableVideoUrl(url: string) {
  return isDirectVideoUrl(url) || !!getEmbedUrl(url);
}

function openUrlVideo(file: Attachment) {
  if (!file.external_url) {
    showToast("URL not available", "error");
    return;
  }

  const directVideo = isDirectVideoUrl(file.external_url);
  const embedUrl = getEmbedUrl(file.external_url);

  if (!directVideo && !embedUrl) {
    window.open(file.external_url, "_blank", "noopener,noreferrer");
    return;
  }

  setVideoPopup({
    title: file.file_name,
    videoUrl: file.external_url,
    embedUrl,
    isDirectVideo: directVideo,
  });
```

```
}

const filteredFiles = files.filter((f) =>
  f.file_name.toLowerCase().includes(search.toLowerCase())
);

// Right side:
// Files assigned to CURRENT selected sidebar client + program
const assigned = filteredFiles.filter((f) => {
  if (!clientId || !programId) return false;

  return (f.assignments || []).some(
    (a) =>
      Number(a.client?.id) === Number(clientId) &&
      Number(a.program?.id) === Number(programId)
  );
});

// Left side:
// Files NOT assigned to CURRENT selected sidebar client + program
const available = filteredFiles.filter((f) => {
  if (!clientId || !programId) {
    // show only files with no exact assignments
    return (f.assignments || []).length === 0;
  }

  const isAssignedToCurrentSelection = (f.assignments || []).some(
    (a) =>
      Number(a.client?.id) === Number(clientId) &&
```

```
      Number(a.program?.id) === Number(programId)
    );

    return !isAssignedToCurrentSelection;
  });

  function FileCard({
    file,
    isAvailableSection,
  }: {
    file: Attachment;
    isAvailableSection: boolean;
  }) {
    const cardRef = useRef<HTMLDivElement>(null);

    const isTrulyUnassigned = (file.assignments || []).length === 0;

    useEffect(() => {
      function handleClickOutside(event: MouseEvent) {
        if (
          openMenu === file.id &&
          cardRef.current &&
          !cardRef.current.contains(event.target as Node)
        ) {
          setOpenMenu(null);
        }
      }

      document.addEventListener("mousedown", handleClickOutside);
```

```
      return () => {
        document.removeEventListener("mousedown",
handleClickOutside);
      };
    }, [openMenu, file.id]);

    return (
      <div
        ref={cardRef}
        className="relative border rounded-xl p-4 bg-white dark:bg-
slate-900 border-slate-200 dark:border-slate-700 shadow-sm
hover:shadow-md transition"
      >
        <button
          className="absolute top-3 right-3 text-slate-500 dark:text-
slate-400"
          onClick={() => setOpenMenu(openMenu === file.id ? null :
file.id)}
        >
          <MoreVertical size={18} />
        </button>

        <p className="font-semibold pr-8 break-words">{file.file_name}
</p>

        {file.description && (
          <DescriptionDisplay
            html={file.description}
            className="text-xs text-slate-500 dark:text-slate-400"
```

```
          />
        )}

        <p className="text-xs">
          Clients: {file.clients.map((c) => c.name).join(", ") || "None"}
        </p>

        <p className="text-xs">
          Programs: {file.programs.map((p) => p.name).join(", ") || "None"}
        </p>

        {file.asset_type === "file" && file.sharepoint_url ? (
          <a
            href={file.sharepoint_url}
            target="_blank"
            rel="noopener noreferrer"
            className="text-xs text-emerald-600 hover:text-emerald-700
dark:text-emerald-400 dark:hover:text-emerald-300"
          >
            View File
          </a>
        ) : file.asset_type === "url" && file.external_url ? (
          isPlayableVideoUrl(file.external_url) ? (
            <button
              onClick={() => openUrlVideo(file)}
              className="mt-2 inline-flex items-center gap-2 px-3 py-1.5
text-xs rounded-lg bg-emerald-600 text-white hover:bg-emerald-700
transition"
            >
```

```
            ▶️ Play Video
          </button>
        ) : (
          <a
            href={file.external_url}
            target="_blank"
            rel="noopener noreferrer"
            className="text-xs text-emerald-600 hover:text-emerald-700
dark:text-emerald-400 dark:hover:text-emerald-300"
          >
            Open URL
          </a>
        )
      ) : (
        <span className="text-xs text-slate-400 dark:text-slate-500">
          No link available
        </span>
      )}

      {openMenu === file.id && (
        <div
          className="
            absolute right-3 top-10
            w-[180px]
            bg-white dark:bg-slate-900
            border border-slate-200 dark:border-slate-700
            p-3 rounded-lg shadow-lg
            space-y-2 z-50
          "
```

```
          >
            {isAvailableSection ? (
              <>
                <button
                  onClick={() => assignFileToCurrentSelection(file)}
                  className="w-full px-3 py-2 text-sm rounded bg-emerald-
600 text-white hover:bg-emerald-700"
                >
                  Assign
                </button>

                {/* Delete only if file is truly global unassigned */}
                {isTrulyUnassigned && (
                  <button
                    onClick={() => requestDeleteFile(file.id)}
                    className="w-full px-3 py-2 text-sm rounded bg-red-600
text-white hover:bg-red-700"
                  >
                    Delete
                  </button>
                )}
              </>
            ) : (
            <>
              <button
                onClick={() => openUpdateModal(file)}
                className="w-full px-3 py-2 text-sm rounded bg-blue-600
text-white hover:bg-blue-700"
              >
```

```
              Update
            </button>

            <button
              onClick={() => unassignFile(file)}
              className="w-full px-3 py-2 text-sm rounded bg-amber-500
text-white hover:bg-amber-600"
            >
              Unassign
            </button>
          </>
        )}
        </div>
      )}
    </div>
  );
}

return (
  <div className="space-y-6">
    <div className="mb-4 flex justify-center">
      <div className="relative w-full max-w-md">
        <svg
          className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4
text-slate-400 dark:text-slate-500"
          fill="none"
          stroke="currentColor"
          viewBox="0 0 24 24"
        >
```

```
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              strokeWidth={2}
              d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z"
            />
          </svg>

          <input
            type="text"
            placeholder="Search files..."
            value={search}
            onChange={(e) => setSearch(e.target.value)}
            className="w-full pl-10 pr-4 py-2 border border-slate-200
dark:border-slate-700 rounded-lg bg-white dark:bg-slate-900 text-
slate-900 dark:text-slate-100 focus:outline-none focus:ring-2
focus:ring-emerald-500"
          />
        </div>
      </div>

      <ConfirmDialog
        open={fileIdToDelete != null}
        title="Delete file?"
        description="This action cannot be undone."
        confirmLabel="Delete"
        cancelLabel="Cancel"
        variant="danger"
        onConfirm={confirmDeleteFile}
```

```
        onCancel={() => setFileIdToDelete(null)}
      />
      <UpdateAttachmentDialog
        open={fileToUpdate != null}
        attachmentName={fileToUpdate?.file_name || ""}
        description={fileToUpdate?.description || ""}
        currentUrl={fileToUpdate?.external_url || null}
        updateUrl={updateUrl}
        setUpdateUrl={setUpdateUrl}
        setUpdateFile={setUpdateFile}
        onClose={() => {
          setFileToUpdate(null);
          setUpdateFile(null);
          setUpdateUrl("");
        }}
        onConfirm={confirmUpdateFile}
        loading={updateLoading}
      />

      <ToastContainer toasts={toasts} onClose={removeToast} />

      <UrlVideoPopup
        open={videoPopup != null}
        title={videoPopup?.title || "Video Player"}
        videoUrl={videoPopup?.videoUrl || null}
        embedUrl={videoPopup?.embedUrl || null}
        isDirectVideo={videoPopup?.isDirectVideo || false}
        onClose={() => setVideoPopup(null)}
      />
```

```
      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
        {/* Available Files */}
        <div className="bg-white dark:bg-slate-900 p-4 rounded-xl
border border-slate-200 dark:border-slate-700">
          <h3 className="mb-3 text-slate-900 dark:text-slate-100 inline-
flex items-center gap-2">
            <FolderOpen className="h-4 w-4 text-slate-500 dark:text-
slate-400" aria-hidden />
            Available Files
          </h3>

          <div className="space-y-3 h-[500px] overflow-y-auto">
            {available.length > 0 ? (
              available.map((file) => (
                <FileCard
                  key={file.id}
                  file={file}
                  isAvailableSection={true}
                />
              ))
            ) : (
              <div className="text-center py-8 text-slate-500 dark:text-
slate-400">
                No available files match your search
              </div>
            )}
          </div>
        </div>
```

```
        {/* Assigned Files */}
        <div className="bg-white dark:bg-slate-900 p-4 rounded-xl
border border-slate-200 dark:border-slate-700">
          <h3 className="mb-3 text-slate-900 dark:text-slate-100 inline-
flex items-center gap-2">
            <CheckCircle2 className="h-4 w-4 text-emerald-600 dark:text-
emerald-400" aria-hidden />
            Assigned Files
          </h3>

          <div className="space-y-3 h-[500px] overflow-y-auto">
            {assigned.length > 0 ? (
              assigned.map((file) => (
                <FileCard
                  key={file.id}
                  file={file}
                  isAvailableSection={false}
                />
              ))
            ) : (
              <div className="text-center py-8 text-slate-500 dark:text-
slate-400">
                {!clientId || !programId
                  ? "Select a client and program from the sidebar to view
assigned files"
                  : "No assigned files match your search"}
              </div>
            )}
```

```
          </div>
        </div>
      </div>
    </div>
  );
}
```

this is my uploadedfileslist.tsx

```
"use client";

import { Loader2 } from "lucide-react";

export default function UpdateAttachmentDialog({
  open,
  attachmentName,
  description,
  currentUrl,
  updateUrl,
  setUpdateUrl,
  setUpdateFile,
  onClose,
  onConfirm,
  loading,
}: {
  open: boolean;
  attachmentName: string;
  description: string;
  currentUrl: string | null;
  updateUrl: string;
  setUpdateUrl: (value: string) => void;
```

```
  setUpdateFile: (file: File | null) => void;
  onClose: () => void;
  onConfirm: () => void;
  loading: boolean;
}) {
  if (!open) return null;

  return (
    <div className="fixed inset-0 z-[100] flex items-center justify-
center bg-black/50 px-4">
      <div className="w-full max-w-lg rounded-2xl bg-white dark:bg-
slate-900 border border-slate-200 dark:border-slate-700 shadow-xl p-6
space-y-4">
        <h2 className="text-lg font-semibold text-slate-900 dark:text-
slate-100">
          Update File Content
        </h2>

        <div>
          <label className="text-sm block mb-1 text-slate-700 dark:text-
slate-300">
            Asset Name
          </label>
          <input
            value={attachmentName}
            readOnly
            className="w-full rounded-lg border border-slate-300
dark:border-slate-600 bg-slate-100 dark:bg-slate-800 px-3 py-2 text-
sm"
```

```
          />
        </div>

        <div>
          <label className="text-sm block mb-1 text-slate-700 dark:text-
slate-300">
            Description
          </label>
          <textarea
            value={description}
            readOnly
            rows={3}
            className="w-full rounded-lg border border-slate-300
dark:border-slate-600 bg-slate-100 dark:bg-slate-800 px-3 py-2 text-
sm"
          />
        </div>

        <div>
          <label className="text-sm block mb-1 text-slate-700 dark:text-
slate-300">
            Select New File (Optional)
          </label>
          <input
            type="file"
            onChange={(e) => setUpdateFile(e.target.files?.[0] || null)}
            className="w-full text-sm text-slate-700 dark:text-slate-300
file:mr-3 file:rounded-md file:border-0 file:bg-slate-100 dark:file:bg-
slate-800 file:px-3 file:py-1.5"
```

```
          />
        </div>

        <div>
          <label className="text-sm block mb-1 text-slate-700 dark:text-
slate-300">
            New External URL (Optional)
          </label>
          <input
            type="url"
            value={updateUrl}
            onChange={(e) => setUpdateUrl(e.target.value)}
            placeholder="https://example.com"
            className="w-full rounded-lg border border-slate-300
dark:border-slate-600 bg-white dark:bg-slate-800 px-3 py-2 text-sm"
          />
          {currentUrl && (
            <p className="mt-1 text-xs text-slate-500 dark:text-slate-400
break-all">
              Current URL: {currentUrl}
            </p>
          )}
        </div>

        <p className="text-xs text-slate-500 dark:text-slate-400">
          Name and assignments will remain unchanged. Only file content
or URL will be replaced.
        </p>
```

```
        <div className="flex justify-end gap-3 pt-2">
          <button
            onClick={onClose}
            disabled={loading}
            className="px-4 py-2 rounded-lg border border-slate-300
dark:border-slate-600 text-sm"
          >
            Cancel
          </button>

          <button
            onClick={onConfirm}
            disabled={loading}
            className="px-4 py-2 rounded-lg bg-blue-600 hover:bg-blue-
700 text-white text-sm flex items-center gap-2 disabled:opacity-60"
          >
            {loading && <Loader2 className="h-4 w-4 animate-spin" />}
            Update
          </button>
        </div>
      </div>
    </div>
  );
}
```

this is updateattachmentdialog.tsx
i am able to update file now but what i want to do is once file is updated i want to show last updated datetime there in whichever file is updated
tell me how to do this step by step proparlly in detail

```
from django.db import models
from django.conf import settings


class SecureAttachment(models.Model):

    ASSET_TYPE_CHOICES = (
        ("file", "File"),
        ("url", "URL"),
    )

    asset_type = models.CharField(
        max_length=50,
        choices=ASSET_TYPE_CHOICES,
        default="file"
    )

    file_name = models.CharField(max_length=255)
    description = models.TextField(blank=True)

    # SharePoint fields (only for file)
    sharepoint_item_id = models.CharField(
        max_length=255,
        unique=True,
        null=True,
        blank=True,
    )
    sharepoint_url = models.URLField(null=True, blank=True)
```

```
    # URL-only field
    external_url = models.URLField(max_length=2000, null=True,
blank=True)

    uploaded_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="uploaded_attachments"
    )

    created_at = models.DateTimeField(auto_now_add=True)

    clients = models.ManyToManyField(
        "client.Client",
        blank=True,
        related_name="attachments"
    )

    programs = models.ManyToManyField(
        "program.Program",
        blank=True,
        related_name="attachments"
    )

    sessions = models.ManyToManyField(
        "program.Session",
```

```
        blank=True,
        related_name="attachments"
    )

    is_active = models.BooleanField(default=True)

    class Meta:
        ordering = ["-created_at"]

    def __str__(self):
        return self.file_name

class SecureAttachmentAssignment(models.Model):
    attachment = models.ForeignKey(
        "attachments.SecureAttachment",
        on_delete=models.CASCADE,
        related_name="assignments"
    )
    client = models.ForeignKey(
        "client.Client",
        on_delete=models.CASCADE,
        related_name="secure_attachment_assignments"
    )
    program = models.ForeignKey(
        "program.Program",
        on_delete=models.CASCADE,
        related_name="secure_attachment_assignments"
    )
    created_at = models.DateTimeField(auto_now_add=True)
```

```
    class Meta:
        unique_together = ("attachment", "client", "program")
        ordering = ["-created_at"]

    def __str__(self):
        return f"{self.attachment.file_name} -> {self.client.name} /
{self.program.name}"
this is my models.py
from rest_framework import serializers

from .models import SecureAttachment, SecureAttachmentAssignment
from client.models import Client
from program.models import Program


#
==============================================
=======
# BASIC LOOKUP SERIALIZERS
#
==============================================
=======
class ClientSerializer(serializers.ModelSerializer):
    class Meta:
        model = Client
        fields = ["id", "name"]
```

```
class ProgramSerializer(serializers.ModelSerializer):
    class Meta:
        model = Program
        fields = ["id", "name", "client"]



#
==============================================
=======
# UPLOAD SERIALIZER
#
==============================================
=======
class SecureAttachmentUploadSerializer(serializers.Serializer):
    name = serializers.CharField(required=False, allow_blank=True)
    description = serializers.CharField(required=False, allow_blank=True)

    external_url = serializers.URLField(
        required=False,
        allow_blank=True,
        max_length=2000
    )

    # Keep temporarily for upload compatibility
    clients = serializers.PrimaryKeyRelatedField(
        queryset=Client.objects.all(),
        many=True,
        required=False
    )
```

```
    programs = serializers.PrimaryKeyRelatedField(
        queryset=Program.objects.all(),
        many=True,
        required=False
    )

    def validate(self, data):
        file = self.context["request"].FILES.get("file")
        external_url = data.get("external_url")
        name = data.get("name")

        if not file and not external_url:
            raise serializers.ValidationError("Provide file or external_url.")

        # URL-only upload requires name
        if external_url and not file and not name:
            raise serializers.ValidationError("Name is required when
uploading external_url.")

        return data


#
==============================================
=======
# ASSIGNMENT SERIALIZERS
#
==============================================
```

```
=======
class
SecureAttachmentAssignmentSerializer(serializers.ModelSerializer):
    client = ClientSerializer(read_only=True)
    program = ProgramSerializer(read_only=True)

    class Meta:
        model = SecureAttachmentAssignment
        fields = ["id", "client", "program", "created_at"]


class SecureAttachmentAssignActionSerializer(serializers.Serializer):
    client_id = serializers.PrimaryKeyRelatedField(
        queryset=Client.objects.all(),
        source="client"
    )
    program_id = serializers.PrimaryKeyRelatedField(
        queryset=Program.objects.all(),
        source="program"
    )

    def validate(self, data):
        client = data["client"]
        program = data["program"]

        # Safety check: selected program must belong to selected client
        if getattr(program, "client_id", None) != client.id:
            raise serializers.ValidationError(
                {"program_id": "Selected program does not belong to
```

```
selected client."}
        )

    return data


#
==========================================
=======
# LIST SERIALIZER
#
==========================================
=======
class SecureAttachmentListSerializer(serializers.ModelSerializer):
    # Legacy support (temporary)
    clients = ClientSerializer(many=True, read_only=True)
    programs = ProgramSerializer(many=True, read_only=True)

    # New exact pair mapping
    assignments = SecureAttachmentAssignmentSerializer(many=True,
read_only=True)

    class Meta:
        model = SecureAttachment
        fields = (
            "id",
            "file_name",
            "description",
            "asset_type",
```

```
        "sharepoint_url",
        "external_url",
        "clients",
        "programs",
        "assignments",
        "created_at",
    )


#
=============================================
=======
# LEGACY PATCH SERIALIZER (temporary)
#
=============================================
=======
class SecureAttachmentUpdateSerializer(serializers.Serializer):
    clients = serializers.PrimaryKeyRelatedField(
        queryset=Client.objects.all(),
        many=True,
        required=False
    )
    programs = serializers.PrimaryKeyRelatedField(
        queryset=Program.objects.all(),
        many=True,
        required=False
    )

class SecureAttachmentReplaceSerializer(serializers.Serializer):
```

```
    external_url = serializers.URLField(
        required=False,
        allow_blank=True,
        max_length=2000
    )

    def validate(self, data):
        request = self.context.get("request")
        file = request.FILES.get("file") if request else None
        external_url = (data.get("external_url") or "").strip()

        if not file and not external_url:
            raise serializers.ValidationError(
                "Provide a new file or external_url."
            )

        if file and external_url:
            raise serializers.ValidationError(
                "Provide either a new file or external_url, not both."
            )

        return data
this is serializers.py
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status, permissions
from django.db import transaction, IntegrityError
from django.shortcuts import get_object_or_404
from rest_framework.parsers import MultiPartParser, FormParser
```

```
from .models import SecureAttachment, SecureAttachmentAssignment
from .serializers import (
    SecureAttachmentUploadSerializer,
    SecureAttachmentListSerializer,
    SecureAttachmentUpdateSerializer,
    SecureAttachmentAssignActionSerializer,
    SecureAttachmentReplaceSerializer,
    ProgramSerializer,
)
from .services.sharepoint import (
    upload_file_to_sharepoint,
    create_view_only_link,
    delete_file_from_sharepoint,
)
from program.models import Program
from django.http import JsonResponse
from django.middleware.csrf import get_token
from django.utils.decorators import method_decorator
from django.views.decorators.csrf import csrf_exempt


def csrf_token_view(request):
    return JsonResponse({
        "csrfToken": get_token(request)
    })


def get_attachment_queryset():
```

```
    return
SecureAttachment.objects.filter(is_active=True).prefetch_related(
        "clients",
        "programs",
        "assignments__client",
        "assignments__program",
    )


@method_decorator(csrf_exempt, name='dispatch')
class SecureAttachmentUploadAPIView(APIView):
    authentication_classes = []
    permission_classes = [permissions.AllowAny]
    parser_classes = [MultiPartParser, FormParser]

    def initial(self, request, *args, **kwargs):
        request._dont_enforce_csrf_checks = True
        super().initial(request, *args, **kwargs)

    def post(self, request):
        try:
            data = request.data.copy()
            uploaded_file = request.FILES.get("file")

            serializer = SecureAttachmentUploadSerializer(
                data=data,
                context={"request": request},
                partial=True
            )
```

```
        if not serializer.is_valid():
            return Response(serializer.errors, status=400)

        validated = serializer.validated_data

        with transaction.atomic():
            # ==============================
            # FILE UPLOAD (SharePoint)
            # ==============================
            if uploaded_file:
                existing_file = SecureAttachment.objects.filter(
                    asset_type="file",
                    file_name=uploaded_file.name,
                    is_active=True
                ).first()

                if existing_file:
                    return Response(
                        {"error": "The file you are trying to upload already
exists."},
                        status=status.HTTP_409_CONFLICT
                    )

                upload_result = upload_file_to_sharepoint(
                    file_stream=uploaded_file.file,
                    filename=uploaded_file.name,
                    content_type=uploaded_file.content_type,
                    file_size=uploaded_file.size,
```

```
                )

                item_id = upload_result["id"]

                existing_item = SecureAttachment.objects.filter(
                    sharepoint_item_id=item_id
                ).first()

                if existing_item:
                    return Response(
                        {"error": "The file you are trying to upload already
exists."},
                        status=status.HTTP_409_CONFLICT
                    )

                view_url = create_view_only_link(item_id)

                attachment = SecureAttachment.objects.create(
                    asset_type="file",
                    file_name=uploaded_file.name,
                    description=validated.get("description", ""),
                    sharepoint_item_id=item_id,
                    sharepoint_url=view_url,
                    uploaded_by=request.user if
request.user.is_authenticated else None,
                )

            # ==============================
            # URL ONLY
```

```
            # ==============================
            else:
                attachment = SecureAttachment.objects.create(
                    asset_type="url",
                    file_name=validated["name"],
                    description=validated.get("description", ""),
                    external_url=validated["external_url"],
                    uploaded_by=request.user if
request.user.is_authenticated else None,
                )

            # TEMPORARY legacy sync
            if "clients" in validated:
                attachment.clients.set(validated["clients"])

            if "programs" in validated:
                attachment.programs.set(validated["programs"])

            # NEW exact pair mapping:
            # only create if exactly one client + one program are supplied
            upload_clients = validated.get("clients", [])
            upload_programs = validated.get("programs", [])

            if len(upload_clients) == 1 and len(upload_programs) == 1:
                selected_client = upload_clients[0]
                selected_program = upload_programs[0]

                if getattr(selected_program, "client_id", None) ==
selected_client.id:
```

```
                SecureAttachmentAssignment.objects.get_or_create(
                    attachment=attachment,
                    client=selected_client,
                    program=selected_program,
                )

            # Re-fetch with prefetch so serializer gets fresh assignments
            attachment = get_attachment_queryset().get(pk=attachment.pk)

            return Response(
                SecureAttachmentListSerializer(attachment).data,
                status=201,
            )

        except IntegrityError:
            return Response(
                {"error": "The file you are trying to upload already exists."},
                status=status.HTTP_409_CONFLICT
            )

        except Exception as e:
            print("ERROR in upload API:", repr(e))
            return Response(
                {"error": str(e)},
                status=status.HTTP_500_INTERNAL_SERVER_ERROR
            )

@method_decorator(csrf_exempt, name='dispatch')
class SecureAttachmentReplaceAPIView(APIView):
```

```
    authentication_classes = []
    permission_classes = [permissions.AllowAny]
    parser_classes = [MultiPartParser, FormParser]

    def initial(self, request, *args, **kwargs):
        request._dont_enforce_csrf_checks = True
        super().initial(request, *args, **kwargs)

    def post(self, request, pk):
        attachment = get_object_or_404(SecureAttachment, pk=pk,
is_active=True)

        serializer = SecureAttachmentReplaceSerializer(
            data=request.data,
            context={"request": request},
            partial=True,
        )

        if not serializer.is_valid():
            return Response(serializer.errors, status=400)

        validated = serializer.validated_data
        uploaded_file = request.FILES.get("file")
        external_url = (validated.get("external_url") or "").strip()

        try:
            with transaction.atomic():
                # Keep old values only for safety/logging if needed
                old_sharepoint_item_id = attachment.sharepoint_item_id
```

```
            old_sharepoint_url = attachment.sharepoint_url

            # ====================================
            # CASE 1: REPLACE WITH NEW FILE
            # ====================================
            if uploaded_file:
                # Optional duplicate name check (recommended)
                # Allow same current attachment, block others with same
active file name
                existing_file = SecureAttachment.objects.filter(
                    asset_type="file",
                    file_name=uploaded_file.name,
                    is_active=True
                ).exclude(pk=attachment.pk).first()

                if existing_file:
                    return Response(
                        {"error": "Another active file with this file name already
exists."},
                        status=status.HTTP_409_CONFLICT
                    )

                upload_result = upload_file_to_sharepoint(
                    file_stream=uploaded_file.file,
                    filename=uploaded_file.name,
                    content_type=uploaded_file.content_type,
                    file_size=uploaded_file.size,
                )
```

```
            new_item_id = upload_result["id"]

            existing_item = SecureAttachment.objects.filter(
                sharepoint_item_id=new_item_id
            ).exclude(pk=attachment.pk).first()

            if existing_item:
                return Response(
                    {"error": "The uploaded SharePoint item already exists
in another attachment."},
                    status=status.HTTP_409_CONFLICT
                )

            new_view_url = create_view_only_link(new_item_id)

            # IMPORTANT:
            # Keep SAME attachment row, SAME id, SAME
assignments
            # Update only content-related fields
            attachment.asset_type = "file"

            # Requirement says keep existing name unchanged
            # So DO NOT set attachment.file_name =
uploaded_file.name
            # Keep old attachment.file_name as-is

            attachment.sharepoint_item_id = new_item_id
            attachment.sharepoint_url = new_view_url
            attachment.external_url = None
```

```
                attachment.save()

                # SAFETY RECOMMENDATION:
                # For v1, do NOT delete old SharePoint file automatically.
                # If you want later, we can add delayed cleanup.
                # old_sharepoint_item_id can be used later if needed.

            # ====================================
            # CASE 2: REPLACE WITH NEW URL
            # ====================================
            elif external_url:
                attachment.asset_type = "url"
                attachment.external_url = external_url
                attachment.sharepoint_item_id = None
                attachment.sharepoint_url = None
                attachment.save()

                # SAFETY RECOMMENDATION:
                # If old attachment was SharePoint file, do NOT auto-
delete for v1.
                # That prevents accidental data loss.

        # Re-fetch with prefetch so serializer gets fresh assignments
        attachment = get_attachment_queryset().get(pk=attachment.pk)

        return Response(
            {
                "status": "replaced",
                "attachment":
```

```
SecureAttachmentListSerializer(attachment).data,
            },
            status=200,
        )

    except IntegrityError:
        return Response(
            {"error": "A conflicting attachment already exists."},
            status=status.HTTP_409_CONFLICT
        )

    except Exception as e:
        print("ERROR in replace API:", repr(e))
        return Response(
            {"error": str(e)},
            status=status.HTTP_500_INTERNAL_SERVER_ERROR
        )
class SecureAttachmentListAPIView(APIView):
    permission_classes = [permissions.AllowAny]

    def get(self, request):
        client_id = request.GET.get("client")
        program_id = request.GET.get("program")

        queryset = get_attachment_queryset()

        # TEMPORARY legacy filter support if query params are used
        if client_id and str(client_id).isdigit():
            queryset = queryset.filter(clients__id=int(client_id))
```

```
        if program_id and str(program_id).isdigit():
            queryset = queryset.filter(programs__id=int(program_id))

        queryset = queryset.distinct()
        serializer = SecureAttachmentListSerializer(queryset, many=True)
        return Response(serializer.data, status=200)


class SecureAttachmentDetailAPIView(APIView):
    permission_classes = [permissions.AllowAny]

    def patch(self, request, pk):
        attachment = get_object_or_404(SecureAttachment, pk=pk,
is_active=True)

        serializer = SecureAttachmentUpdateSerializer(data=request.data,
partial=True)
        if not serializer.is_valid():
            return Response(serializer.errors, status=400)

        validated = serializer.validated_data

        if "clients" in validated:
            attachment.clients.set(validated["clients"])

        if "programs" in validated:
            attachment.programs.set(validated["programs"])
```

```
        attachment.save()

        # Re-fetch prefetched version
        attachment = get_attachment_queryset().get(pk=attachment.pk)

        return Response(
            SecureAttachmentListSerializer(attachment).data,
            status=200,
        )

    def delete(self, request, pk):
        attachment = get_object_or_404(SecureAttachment, pk=pk)

        try:
            with transaction.atomic():
                # If it's a SharePoint file, delete from SharePoint first
                if attachment.asset_type == "file" and
attachment.sharepoint_item_id:

delete_file_from_sharepoint(attachment.sharepoint_item_id)

                # Hard delete from DB
                attachment.delete()

            return Response({"status": "deleted"}, status=200)

        except Exception as e:
            return Response(
                {"error": f"Delete failed: {str(e)}"},
```

```
            status=status.HTTP_500_INTERNAL_SERVER_ERROR
        )


class SecureAttachmentAssignAPIView(APIView):
    permission_classes = [permissions.AllowAny]

    def post(self, request, pk):
        attachment = get_object_or_404(SecureAttachment, pk=pk,
is_active=True)

        serializer =
SecureAttachmentAssignActionSerializer(data=request.data)
        if not serializer.is_valid():
            return Response(serializer.errors, status=400)

        validated = serializer.validated_data
        client = validated["client"]
        program = validated["program"]

        # Create exact pair assignment
        assignment, created =
SecureAttachmentAssignment.objects.get_or_create(
            attachment=attachment,
            client=client,
            program=program,
        )

        # TEMPORARY: keep legacy M2M fields in sync for display
```

```
compatibility
        attachment.clients.add(client)
        attachment.programs.add(program)

        # Re-fetch prefetched version
        attachment = get_attachment_queryset().get(pk=attachment.pk)

        return Response(
            {
                "status": "assigned",
                "created": created,
                "attachment":
SecureAttachmentListSerializer(attachment).data,
            },
            status=200,
        )


class SecureAttachmentUnassignAPIView(APIView):
    permission_classes = [permissions.AllowAny]

    def post(self, request, pk):
        attachment = get_object_or_404(SecureAttachment, pk=pk,
is_active=True)

        serializer =
SecureAttachmentAssignActionSerializer(data=request.data)
        if not serializer.is_valid():
            return Response(serializer.errors, status=400)
```

```
        validated = serializer.validated_data
        client = validated["client"]
        program = validated["program"]

        deleted_count, _ = SecureAttachmentAssignment.objects.filter(
            attachment=attachment,
            client=client,
            program=program,
        ).delete()

        # TEMPORARY: rebuild legacy M2M fields from remaining exact
assignments
        remaining_assignments =
SecureAttachmentAssignment.objects.filter(
            attachment=attachment
        )

        remaining_client_ids = list(
            remaining_assignments.values_list("client_id",
flat=True).distinct()
        )
        remaining_program_ids = list(
            remaining_assignments.values_list("program_id",
flat=True).distinct()
        )

        attachment.clients.set(remaining_client_ids)
        attachment.programs.set(remaining_program_ids)
```

```
        # Re-fetch prefetched version
        attachment = get_attachment_queryset().get(pk=attachment.pk)

        return Response(
            {
                "status": "unassigned",
                "deleted": deleted_count > 0,
                "attachment":
SecureAttachmentListSerializer(attachment).data,
            },
            status=200,
        )


class ProgramsByClientAPIView(APIView):
    permission_classes = [permissions.AllowAny]

    def get(self, request):
        client_id = request.GET.get("client")

        if not client_id:
            return Response([], status=200)

        programs = Program.objects.filter(client_id=client_id)
        serializer = ProgramSerializer(programs, many=True)
        return Response(serializer.data)
```

views.py

no tell me accordinglly use django inbuilt function

Perfect 👍
Since **you are already updating the same** `SecureAttachment` **row** (same `id`) in `SecureAttachmentReplaceAPIView`, the **best Django built-in way** is:

- add a `updated_at = models.DateTimeField(auto_now=True)`
- include it in serializer
- send it to frontend
- show it in `UploadedFilesList.tsx`

This is the **cleanest + Django built-in** solution.

✅ What you want

Whenever a file is updated (either:

- replaced with a **new file**, or
- replaced with a **new URL**)

you want to display:

- **Last Updated: date + time**

inside each file card in `uploadedfileslist.tsx`.

✅ Best Django built-in solution

Use Django's built-in:

Python Run

Why this is best:

- automatically updates **every time** `.save()` **is called**
- no manual timestamp logic needed
- works perfectly with your existing `attachment.save()` in replace API
- future-proof for any other updates too

🚀 STEP-BY-STEP (PROPERLY)

STEP 1) Update your Django model (`models.py`)

Open:

`attachments/models.py`

Inside `SecureAttachment` model, add this field:

Python Run

✅ Your updated `SecureAttachment` model should look like this

> I am showing only the relevant part with exact placement:

```
class SecureAttachment(models.Model):

    ASSET_TYPE_CHOICES = (
        ("file", "File"),
        ("url", "URL"),
    )

    asset_type = models.CharField(
        max_length=50,
        choices=ASSET_TYPE_CHOICES,
        default="file"
    )

    file_name = models.CharField(max_length=255)
    description = models.TextField(blank=True)

    # SharePoint fields (only for file)
    sharepoint_item_id = models.CharField(
        max_length=255,
        unique=True,
        null=True,
        blank=True,
```

STEP 2) Create migration

Run in terminal:

Bash

You will get a migration for `updated_at`.

STEP 3) Apply migration

Run:

Bash

⚠️ Important note about existing old records

For existing rows, Django may ask for a default during migration because `updated_at` is non-nullable.

If it asks:

- choose **option 1**
- provide a one-time default like:

Python Run

But **better** approach is to define it like this first:

```python
from django.utils import timezone
```

Python Run

Then later you can keep it nullable (totally fine for your case).

Recommended for smooth migration:

Use this in model first:

Python Run

This avoids migration trouble for old records.

✅ Recommended final field (safer for your current project)

Use this version:

Python Run

STEP 4) Expose `updated_at` in serializer (`serializers.py`)

Open:

`attachments/serializers.py`

In:

Python Run

Add `"updated_at"` to `fields`.

Replace this:

```python
fields = (
    "id",
    "file_name",
    "description",
    "asset_type",
    "sharepoint_url",
    "external_url",
    "clients",
    "programs",
    "assignments",
    "created_at",
)
```

With this:

```python
fields = (
    "id",
    "file_name",
    "description",
    "asset_type",
    "sharepoint_url",
    "external_url",
    "clients",
    "programs",
    "assignments",
    "created_at",
    "updated_at",
)
```

STEP 5) No need to change replace API logic (very important)

Your existing replace API already does:

Python Run

inside both cases:

- file replacement
- URL replacement

Because `updated_at` uses:

Python Run

Django will **automatically update** `updated_at` **on every save**.

✅ That means your existing code already works

For example:

```
attachment.asset_type = "file"
attachment.sharepoint_item_id = new_item_id
attachment.sharepoint_url = new_view_url
attachment.external_url = None
attachment.save()
```

and

```
attachment.asset_type = "url"
attachment.external_url = external_url
attachment.sharepoint_item_id = None
attachment.sharepoint_url = None
attachment.save()
```

Both will automatically update:

Python Run

✅ No extra backend logic needed.

STEP 6) Update frontend `Attachment` type in `uploadedfileslist.tsx`

Open:

`uploadedfileslist.tsx`

Find:

```
type Attachment = {
  id: number;
  file_name: string;
  description: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
  clients: Option[];
  programs: ProgramOption[];
  assignments: Assignment[];
};
```

Replace with:

```
type Attachment = {
  id: number;
  file_name: string;
  description: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
  clients: Option[];
  programs: ProgramOption[];
  assignments: Assignment[];
  created_at?: string;
  updated_at?: string | null;
};
```

STEP 7) Update `normalizeAttachment()` in `uploadedfileslist.tsx`

Find your current function:

```
function normalizeAttachment(raw: any): Attachment {
  return {
    id: Number(raw?.id),
    file_name: raw?.file_name || "",
    description: raw?.description || "",
    asset_type: raw?.asset_type || "file",
    sharepoint_url: raw?.sharepoint_url || null,
    external_url: raw?.external_url || null,
    clients: Array.isArray(raw?.clients) ? raw.clients : [],
    programs: Array.isArray(raw?.programs) ? raw.programs : [],
```

```
    assignments: Array.isArray(raw?.assignments) ? raw.assignments : [],
  };
}
```

Replace with:

```
function normalizeAttachment(raw: any): Attachment {
  return {
    id: Number(raw?.id),
    file_name: raw?.file_name || "",
    description: raw?.description || "",
    asset_type: raw?.asset_type || "file",
    sharepoint_url: raw?.sharepoint_url || null,
    external_url: raw?.external_url || null,
    clients: Array.isArray(raw?.clients) ? raw.clients : [],
    programs: Array.isArray(raw?.programs) ? raw.programs : [],
    assignments: Array.isArray(raw?.assignments) ? raw.assignments : [],
    created_at: raw?.created_at || undefined,
    updated_at: raw?.updated_at || null,
  };
}
```

STEP 8) Add a helper function to format datetime nicely

In `uploadedfileslist.tsx`, add this function **above** `const filteredFiles = ...` (or anywhere inside component before return).

Add this:

```
function formatDateTime(value?: string | null) {
  if (!value) return "Never";

  try {
    return new Date(value).toLocaleString("en-IN", {
      dateStyle: "medium",
      timeStyle: "short",
    });
  } catch {
    return value;
  }
}
```

STEP 9) Show “Last Updated” in each file card

Inside your `FileCard` component, you currently have:

```
<p className="font-semibold pr-8 break-words">{file.file_name}</p>

{file.description && (
  <DescriptionDisplay
    html={file.description}
    className="text-xs text-slate-500 dark:text-slate-400"
  />
)}
```

```
<p className="text-xs">
  Clients: {file.clients.map((c) => c.name).join(", ") || "None"}
</p>

<p className="text-xs">
  Programs: {file.programs.map((p) => p.name).join(", ") || "None"}
</p>
```

Add this block below Programs (recommended place)

```
<p className="text-xs text-slate-500 dark:text-slate-400">
  Last Updated: {formatDateTime(file.updated_at || file.created_at)}
```

TypeScript

Final section will look like this:

TypeScript

```
<p className="font-semibold pr-8 break-words">{file.file_name}</p>

{file.description && (
  <DescriptionDisplay
    html={file.description}
    className="text-xs text-slate-500 dark:text-slate-400"
  />
)}
```

```
<p className="text-xs">
  Clients: {file.clients.map((c) => c.name).join(", ") || "None"}
</p>

<p className="text-xs">
  Programs: {file.programs.map((p) => p.name).join(", ") || "None"}
</p>

<p className="text-xs text-slate-500 dark:text-slate-400">
  Last Updated: {formatDateTime(file.updated_at || file.created_at)}
</p>
```

✅ Why `file.updated_at || file.created_at` is better

Because:

- For **newly uploaded files**, `updated_at` may be same as created_at
- For **old existing records**, if `updated_at` is null (because of old data), fallback to `created_at`
- Prevents blank UI

STEP 10) Done — no changes needed in `UpdateAttachmentDialog.tsx`

Your dialog component does **not** need any changes.

Because:

- update happens in backend
- backend returns updated attachment
- `confirmUpdateFile()` already updates state + refetches

- UI will automatically show new `updated_at`

✅ Final flow after this

When user clicks **Update**:

1. `confirmUpdateFile()` calls:

```typescript
POST /api/attachments/{id}/replace/
```

2. Django updates same row:

```python
attachment.save()
```

3. Because of:

```python
updated_at = models.DateTimeField(auto_now=True)
```

   Django automatically updates timestamp

4. Serializer returns:

```json
{
  "id": 12,
```

```
  "file_name": "abc.pdf",
  "updated_at": "2026-03-27T17:30:10.123456+05:30"
}
```

5. Frontend `normalizeAttachment()` stores it
6. File card shows:

```typescript
Last Updated: 27 Mar 2026, 5:30 pm
```

🔥 Exact minimal code changes summary

---

1) `models.py`

Add:

```python
updated_at = models.DateTimeField(auto_now=True, null=True, blank=True)
```

---

2) Run:

```bash
```

```
python manage.py makemigrations
python manage.py migrate
```

---

3) `serializers.py`

In `SecureAttachmentListSerializer` add:

```python
"updated_at",
```

---

4) `uploadedfileslist.tsx` type

Add:

```typescript
created_at?: string;
updated_at?: string | null;
```

---

5) `normalizeAttachment()`

Add:

```typescript
```

```
created_at: raw?.created_at || undefined,
updated_at: raw?.updated_at || null,
```

---

6) Add helper:

TypeScript

```typescript
function formatDateTime(value?: string | null) {
  if (!value) return "Never";

  try {
    return new Date(value).toLocaleString("en-IN", {
      dateStyle: "medium",
      timeStyle: "short",
    });
  } catch {
    return value;
  }
}
```

---

7) Show in UI:

TypeScript

```typescript
<p className="text-xs text-slate-500 dark:text-slate-400">
  Last Updated: {formatDateTime(file.updated_at || file.created_at)}
</p>
```

✅ Best practice suggestion (optional but recommended)

Right now your model ordering is:

```python
class Meta:
    ordering = ["-created_at"]
```

If later you want **recently updated files first**, change to:

```python
class Meta:
    ordering = ["-updated_at", "-created_at"]
```

But for now:

Keep as is unless you want update-based sorting.

🎯 Important note about your current replace logic

You are already doing the right thing:

- same attachment row
- same `id`
- same assignments
- only content changes

That means `updated_at` is **exactly the correct field** for this use case.

✅ My recommendation for you

Use this exact model field:

Python Run

```python
updated_at = models.DateTimeField(auto_now=True, null=True, blank=True)
```

This is safest for your current DB because you already have existing attachments.

If you want, I can now give you the **exact copy-paste final updated code blocks** for:

1. **models.py** (full `SecureAttachment` class)
2. **serializers.py** (`SecureAttachmentListSerializer`)
3. **uploadedfileslist.tsx** (exact replaceable snippets in correct locations)

👉 If you want, I'll give you **copy-paste ready code only** (no extra explanation) so you can directly replace it.

"use client";

import { useEffect, useState, useRef } from "react";
import { MoreVertical, FolderOpen, CheckCircle2 } from "lucide-react";
import { useToast, ToastContainer } from "@/components/Toast";
import { ConfirmDialog } from "@/components/ConfirmDialog";
import DescriptionDisplay from "@/components/DescriptionDisplay";
import UrlVideoPopup from "@/components/UrlVideoPopup";

```
import UpdateAttachmentDialog from
"@/components/UpdateAttachmentDialog";

type Option = { id: number; name: string };

type ProgramOption = {
  id: number;
  name: string;
  client?: number;
};

type Assignment = {
  id: number;
  client: Option;
  program: ProgramOption;
  created_at?: string;
};

type Attachment = {
  id: number;
  file_name: string;
  description: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
  clients: Option[];
  programs: ProgramOption[];
  assignments: Assignment[];
  created_at?: string;
```

```
  updated_at?: string | null;
};

export default function UploadedFilesList({
  refreshKey,
  clientId,
  programId,
}: {
  refreshKey?: number;
  clientId?: number | null;
  programId?: number | null;
}) {
  const API = process.env.NEXT_PUBLIC_API_BASE_URL;

  const [files, setFiles] = useState<Attachment[]>([]);
  const [openMenu, setOpenMenu] = useState<number | null>(null);
  const [search, setSearch] = useState("");
  const [fileIdToDelete, setFileIdToDelete] = useState<number | null>
(null);
  const [videoPopup, setVideoPopup] = useState<{
    title: string;
    videoUrl: string;
    embedUrl: string | null;
    isDirectVideo: boolean;
  } | null>(null);
  const [fileToUpdate, setFileToUpdate] = useState<Attachment | null>
(null);
  const [updateFile, setUpdateFile] = useState<File | null>(null);
  const [updateUrl, setUpdateUrl] = useState("");
```

```
const [updateLoading, setUpdateLoading] = useState(false);

const { toasts, showToast, removeToast } = useToast();
  /** 1GB limit */
const MAX_FILE_SIZE = 1024 * 1024 * 1024;

/** Allowed MIME prefixes & types */
const ALLOWED_TYPES = [
  "image/",
  "application/pdf",
  "application/msword",
  "application/vnd.ms-excel",
  "application/vnd.ms-powerpoint",
  "application/vnd.openxmlformats-officedocument",
  "text/plain",
  "video/",
  "audio/",
];

function validateFile(file: File) {
  if (file.size > MAX_FILE_SIZE) {
    return "File size exceeds 1GB limit";
  }

  const isAllowed = ALLOWED_TYPES.some((type) =>
    file.type.startsWith(type)
  );

  if (!isAllowed) {
```

```
      return "Unsupported file type";
    }

    return null;
  }

  function openUpdateModal(file: Attachment) {
    setOpenMenu(null);
    setFileToUpdate(file);
    setUpdateFile(null);
    setUpdateUrl(file.external_url || "");
  }
  function notifyAttachmentsUpdated() {
    localStorage.setItem("attachments_updated_at",
Date.now().toString());
    window.dispatchEvent(new Event("attachments-updated"));
  }

  function normalizeAttachment(raw: any): Attachment {
    return {
      id: Number(raw?.id),
      file_name: raw?.file_name || "",
      description: raw?.description || "",
      asset_type: raw?.asset_type || "file",
      sharepoint_url: raw?.sharepoint_url || null,
      external_url: raw?.external_url || null,
      clients: Array.isArray(raw?.clients) ? raw.clients : [],
      programs: Array.isArray(raw?.programs) ? raw.programs : [],
      assignments: Array.isArray(raw?.assignments) ? raw.assignments : [],
```

```
    created_at: raw?.created_at || undefined,
    updated_at: raw?.updated_at || null,
  };
}

async function fetchFilesREST() {
  try {
    const res = await fetch(`${API}/api/attachments/`, {
      cache: "no-store",
    });

    const data = await res.json().catch(() => []);

    if (!res.ok) {
      throw new Error(data?.error || "Failed to fetch attachments");
    }

    const normalized = Array.isArray(data)
      ? data.map(normalizeAttachment)
      : [];

    // TEMP DEBUG - remove later if you want
    console.log("ATTACHMENTS API DATA:", normalized);

    setFiles(normalized);
  } catch (err: any) {
    console.error("Failed to fetch attachments:", err);
    showToast(err.message || "Failed to load files", "error");
    setFiles([]);
```

```
    }
  }

  useEffect(() => {
    fetchFilesREST();
  }, [refreshKey]);

  // IMPORTANT: listen for global updates from other components
  useEffect(() => {
    function handleAttachmentsUpdated() {
      fetchFilesREST();
    }

    window.addEventListener("attachments-updated",
handleAttachmentsUpdated);
    return () => {
      window.removeEventListener("attachments-updated",
handleAttachmentsUpdated);
    };
  }, []);

  async function assignFileToCurrentSelection(file: Attachment) {
    if (!clientId || !programId) {
      showToast("Please select a Client and Program from the sidebar
first", "error");
      return;
    }

    try {
```

```
    const res = await
fetch(`${API}/api/attachments/${file.id}/assign/`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        client_id: Number(clientId),
        program_id: Number(programId),
      }),
    });

    const data = await res.json().catch(() => ({}));

    if (!res.ok) {
      throw new Error(
        data?.program_id?.[0] ||
          data?.non_field_errors?.[0] ||
          data?.error ||
          "Failed to assign file"
      );
    }

    const updatedAttachment = data?.attachment
      ? normalizeAttachment(data.attachment)
      : null;

    // Update local state immediately for instant UI response
    if (updatedAttachment) {
      setFiles((prev) =>
        prev.map((f) => (f.id === file.id ? updatedAttachment : f))
```

```
      );
    }

    setOpenMenu(null);

    showToast(
      data?.created === false
        ? "File already assigned to selected client/program"
        : "File assigned successfully",
      "success"
    );

    notifyAttachmentsUpdated();

    // SAFEST: force re-fetch so assigned section is always correct
    await fetchFilesREST();
  } catch (err: any) {
    showToast(err.message || "Assignment failed", "error");
  }
}

async function unassignFile(file: Attachment) {
  if (!clientId || !programId) {
    showToast("Please select a Client and Program first", "error");
    return;
  }
```

ChatGPT Share •••

```
    const res = await
```

```
fetch(`${API}/api/attachments/${file.id}/unassign/`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      client_id: Number(clientId),
      program_id: Number(programId),
    }),
  });

  const data = await res.json().catch(() => ({}));

  if (!res.ok) {
    throw new Error(
      data?.program_id?.[0] ||
        data?.non_field_errors?.[0] ||
        data?.error ||
        "Failed to unassign file"
    );
  }

  const updatedAttachment = data?.attachment
    ? normalizeAttachment(data.attachment)
    : null;

  // Update local state immediately for instant UI response
  if (updatedAttachment) {
    setFiles((prev) =>
      prev.map((f) => (f.id === file.id ? updatedAttachment : f))
    );
```

```
    }

    setOpenMenu(null);

    showToast(
      data?.deleted
        ? "File unassigned successfully"
        : "File was already unassigned",
      "success"
    );

    notifyAttachmentsUpdated();

    // SAFEST: force re-fetch so assigned section is always correct
    await fetchFilesREST();
  } catch (err: any) {
    showToast(err.message || "Unassign failed", "error");
  }
}
async function confirmUpdateFile() {
  if (!fileToUpdate) return;

  if (!updateFile && !updateUrl.trim()) {
    showToast("Please select a new file or provide a new URL", "error");
    return;
  }

  // Prevent ambiguity
  if (updateFile && updateUrl.trim()) {
```

```
      showToast("Please provide either a new file or a new URL, not
both", "error");
      return;
    }

    if (updateFile) {
      const validationError = validateFile(updateFile);
      if (validationError) {
        showToast(validationError, "error");
        return;
      }
    }

    setUpdateLoading(true);

    try {
      const fd = new FormData();

      if (updateFile) {
        fd.append("file", updateFile);
      }

      if (updateUrl.trim()) {
        fd.append("external_url", updateUrl.trim());
      }

      // Keep same existing values
      fd.append("name", fileToUpdate.file_name);
      fd.append("description", fileToUpdate.description || "");
```

```
    const res = await fetch(
      `${API}/api/attachments/${fileToUpdate.id}/replace/`,
      {
        method: "POST",
        body: fd,
        credentials: "include",
      }
    );

    const data = await res.json().catch(() => ({}));

    if (!res.ok) {
      throw new Error(data?.error || "Failed to update file");
    }

    const updatedAttachment = data?.attachment
      ? normalizeAttachment(data.attachment)
      : null;

    // instant UI refresh
    if (updatedAttachment) {
      setFiles((prev) =>
        prev.map((f) => (f.id === updatedAttachment.id ?
updatedAttachment : f))
      );
    }

    // close modal + reset state
```

```
    setFileToUpdate(null);
    setUpdateFile(null);
    setUpdateUrl("");

    showToast("File updated successfully", "success");

    notifyAttachmentsUpdated();

    // safest final refresh
    await fetchFilesREST();
  } catch (err: any) {
    showToast(err.message || "Update failed", "error");
  } finally {
    setUpdateLoading(false);
  }
}

function requestDeleteFile(fileId: number) {
  setOpenMenu(null);
  setFileIdToDelete(fileId);
}

async function confirmDeleteFile() {
  if (fileIdToDelete == null) return;

  const fileId = fileIdToDelete;
  setFileIdToDelete(null);

  try {
```

```
    const res = await fetch(`${API}/api/attachments/${fileId}/`, {
      method: "DELETE",
    });

    const data = await res.json().catch(() => ({}));

    if (!res.ok) {
      throw new Error(data?.error || "Failed to delete file");
    }

    setFiles((prev) => prev.filter((f) => f.id !== fileId));
    showToast("File deleted successfully", "success");
    notifyAttachmentsUpdated();

    // optional safety refetch
    await fetchFilesREST();
  } catch (err: any) {
    showToast(err.message || "Delete failed", "error");
  }
}

function isDirectVideoUrl(url: string) {
  const lower = url.toLowerCase();
  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v")
```

```
    );
  }

  function getEmbedUrl(url: string): string | null {
    try {
      const parsed = new URL(url);
      const host = parsed.hostname.toLowerCase();

      // YouTube short link: youtu.be/VIDEO_ID
      if (host.includes("youtu.be")) {
        const videoId = parsed.pathname.replace("/", "");
        return videoId ? `https://www.youtube.com/embed/${videoId}` :
null;
      }

      // YouTube normal link: youtube.com/watch?v=VIDEO_ID
      if (host.includes("youtube.com")) {
        const videoId = parsed.searchParams.get("v");
        if (videoId) return `https://www.youtube.com/embed/${videoId}`;

        // already embed url
        if (parsed.pathname.includes("/embed/")) return url;
      }

      // Vimeo
      if (host.includes("vimeo.com")) {
        const pathParts = parsed.pathname.split("/").filter(Boolean);
        const videoId = pathParts[pathParts.length - 1];
        return videoId ? `https://player.vimeo.com/video/${videoId}` :
```

```
null;
      }

      // Loom
      if (host.includes("loom.com")) {
        const pathParts = parsed.pathname.split("/").filter(Boolean);

        const shareIndex = pathParts.findIndex((part) => part ===
"share");
        if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
          return https://www.loom.com/embed/${pathParts[shareIndex +
1]};
        }

        const embedIndex = pathParts.findIndex((part) => part ===
"embed");
        if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
          return url;
        }
      }

      return null;
    } catch {
      return null;
    }
  }

  function isPlayableVideoUrl(url: string) {
    return isDirectVideoUrl(url) || !!getEmbedUrl(url);
```

```
}

function openUrlVideo(file: Attachment) {
  if (!file.external_url) {
    showToast("URL not available", "error");
    return;
  }

  const directVideo = isDirectVideoUrl(file.external_url);
  const embedUrl = getEmbedUrl(file.external_url);

  if (!directVideo && !embedUrl) {
    window.open(file.external_url, "_blank", "noopener,noreferrer");
    return;
  }

  setVideoPopup({
    title: file.file_name,
    videoUrl: file.external_url,
    embedUrl,
    isDirectVideo: directVideo,
  });
}

function formatDateTime(value?: string | null) {
  if (!value) return "Never";

  try {
    return new Date(value).toLocaleString("en-IN", {
```

```
      dateStyle: "medium",
      timeStyle: "short",
    });
  } catch {
    return value;
  }
}

const filteredFiles = files.filter((f) =>
  f.file_name.toLowerCase().includes(search.toLowerCase())
);

// Right side:
// Files assigned to CURRENT selected sidebar client + program
const assigned = filteredFiles.filter((f) => {
  if (!clientId || !programId) return false;

  return (f.assignments || []).some(
    (a) =>
      Number(a.client?.id) === Number(clientId) &&
      Number(a.program?.id) === Number(programId)
  );
});

// Left side:
// Files NOT assigned to CURRENT selected sidebar client + program
const available = filteredFiles.filter((f) => {
  if (!clientId || !programId) {
    // show only files with no exact assignments
```

```
    return (f.assignments || []).length === 0;
  }

  const isAssignedToCurrentSelection = (f.assignments || []).some(
    (a) =>
      Number(a.client?.id) === Number(clientId) &&
      Number(a.program?.id) === Number(programId)
  );

  return !isAssignedToCurrentSelection;
});

function FileCard({
  file,
  isAvailableSection,
}: {
  file: Attachment;
  isAvailableSection: boolean;
}) {
  const cardRef = useRef<HTMLDivElement>(null);

  const isTrulyUnassigned = (file.assignments || []).length === 0;

  useEffect(() => {
    function handleClickOutside(event: MouseEvent) {
      if (
        openMenu === file.id &&
        cardRef.current &&
        !cardRef.current.contains(event.target as Node)
```

```
        ) {
          setOpenMenu(null);
        }
      }

      document.addEventListener("mousedown", handleClickOutside);
      return () => {
        document.removeEventListener("mousedown",
handleClickOutside);
      };
    }, [openMenu, file.id]);

    return (
      <div
        ref={cardRef}
        className="relative border rounded-xl p-4 bg-white dark:bg-
slate-900 border-slate-200 dark:border-slate-700 shadow-sm
hover:shadow-md transition"
      >
        <button
          className="absolute top-3 right-3 text-slate-500 dark:text-
slate-400"
          onClick={() => setOpenMenu(openMenu === file.id ? null :
file.id)}
        >
          <MoreVertical size={18} />
        </button>

        <p className="font-semibold pr-8 break-words">{file.file_name}
```

```
</p>

        {file.description && (
          <DescriptionDisplay
            html={file.description}
            className="text-xs text-slate-500 dark:text-slate-400"
          />
        )}

        <p className="text-xs">
          Clients: {file.clients.map((c) => c.name).join(", ") || "None"}
        </p>

        <p className="text-xs">
          Programs: {file.programs.map((p) => p.name).join(", ") || "None"}
        </p>

        <p className="text-xs text-slate-500 dark:text-slate-400">
          Last Updated: {formatDateTime(file.updated_at || file.created_at)}
        </p>

        {file.asset_type === "file" && file.sharepoint_url ? (
          <a
            href={file.sharepoint_url}
            target="_blank"
            rel="noopener noreferrer"
            className="text-xs text-emerald-600 hover:text-emerald-700
dark:text-emerald-400 dark:hover:text-emerald-300"
          >
```

```
          View File
        </a>
      ) : file.asset_type === "url" && file.external_url ? (
        isPlayableVideoUrl(file.external_url) ? (
          <button
            onClick={() => openUrlVideo(file)}
            className="mt-2 inline-flex items-center gap-2 px-3 py-1.5
text-xs rounded-lg bg-emerald-600 text-white hover:bg-emerald-700
transition"
          >
            ▶️ Play Video
          </button>
        ) : (
          <a
            href={file.external_url}
            target="_blank"
            rel="noopener noreferrer"
            className="text-xs text-emerald-600 hover:text-emerald-700
dark:text-emerald-400 dark:hover:text-emerald-300"
          >
            Open URL
          </a>
        )
      ) : (
        <span className="text-xs text-slate-400 dark:text-slate-500">
          No link available
        </span>
      )}
```

```
        {openMenu === file.id && (
          <div
            className="
              absolute right-3 top-10
              w-[180px]
              bg-white dark:bg-slate-900
              border border-slate-200 dark:border-slate-700
              p-3 rounded-lg shadow-lg
              space-y-2 z-50
            "
          >
            {isAvailableSection ? (
              <>
                <button
                  onClick={() => assignFileToCurrentSelection(file)}
                  className="w-full px-3 py-2 text-sm rounded bg-emerald-
600 text-white hover:bg-emerald-700"
                >
                  Assign
                </button>

                {/* Delete only if file is truly global unassigned */}
                {isTrulyUnassigned && (
                  <button
                    onClick={() => requestDeleteFile(file.id)}
                    className="w-full px-3 py-2 text-sm rounded bg-red-600
text-white hover:bg-red-700"
                  >
                    Delete
```

```
                </button>
              )}
            </>
          ) : (
          <>
            <button
              onClick={() => openUpdateModal(file)}
              className="w-full px-3 py-2 text-sm rounded bg-blue-600
text-white hover:bg-blue-700"
            >
              Update
            </button>

            <button
              onClick={() => unassignFile(file)}
              className="w-full px-3 py-2 text-sm rounded bg-amber-500
text-white hover:bg-amber-600"
            >
              Unassign
            </button>
          </>
        )}
        </div>
      )}
    </div>
  );
}

return (
```

```
    <div className="space-y-6">
      <div className="mb-4 flex justify-center">
        <div className="relative w-full max-w-md">
          <svg
            className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4
text-slate-400 dark:text-slate-500"
            fill="none"
            stroke="currentColor"
            viewBox="0 0 24 24"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              strokeWidth={2}
              d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z"
            />
          </svg>

          <input
            type="text"
            placeholder="Search files..."
            value={search}
            onChange={(e) => setSearch(e.target.value)}
            className="w-full pl-10 pr-4 py-2 border border-slate-200
dark:border-slate-700 rounded-lg bg-white dark:bg-slate-900 text-
slate-900 dark:text-slate-100 focus:outline-none focus:ring-2
focus:ring-emerald-500"
          />
        </div>
```

```
      </div>

      <ConfirmDialog
        open={fileIdToDelete != null}
        title="Delete file?"
        description="This action cannot be undone."
        confirmLabel="Delete"
        cancelLabel="Cancel"
        variant="danger"
        onConfirm={confirmDeleteFile}
        onCancel={() => setFileIdToDelete(null)}
      />
      <UpdateAttachmentDialog
        open={fileToUpdate != null}
        attachmentName={fileToUpdate?.file_name || ""}
        description={fileToUpdate?.description || ""}
        currentUrl={fileToUpdate?.external_url || null}
        updateUrl={updateUrl}
        setUpdateUrl={setUpdateUrl}
        setUpdateFile={setUpdateFile}
        onClose={() => {
          setFileToUpdate(null);
          setUpdateFile(null);
          setUpdateUrl("");
        }}
        onConfirm={confirmUpdateFile}
        loading={updateLoading}
      />
```

```
      <ToastContainer toasts={toasts} onClose={removeToast} />

      <UrlVideoPopup
        open={videoPopup != null}
        title={videoPopup?.title || "Video Player"}
        videoUrl={videoPopup?.videoUrl || null}
        embedUrl={videoPopup?.embedUrl || null}
        isDirectVideo={videoPopup?.isDirectVideo || false}
        onClose={() => setVideoPopup(null)}
      />

      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
        {/* Available Files */}
        <div className="bg-white dark:bg-slate-900 p-4 rounded-xl
border border-slate-200 dark:border-slate-700">
          <h3 className="mb-3 text-slate-900 dark:text-slate-100 inline-
flex items-center gap-2">
            <FolderOpen className="h-4 w-4 text-slate-500 dark:text-
slate-400" aria-hidden />
            Available Files
          </h3>

          <div className="space-y-3 h-[500px] overflow-y-auto">
            {available.length > 0 ? (
              available.map((file) => (
                <FileCard
                  key={file.id}
                  file={file}
                  isAvailableSection={true}
```

```
            />
          ))
        ) : (
          <div className="text-center py-8 text-slate-500 dark:text-
slate-400">
            No available files match your search
          </div>
        )}
      </div>
    </div>

    {/* Assigned Files */}
    <div className="bg-white dark:bg-slate-900 p-4 rounded-xl
border border-slate-200 dark:border-slate-700">
      <h3 className="mb-3 text-slate-900 dark:text-slate-100 inline-
flex items-center gap-2">
        <CheckCircle2 className="h-4 w-4 text-emerald-600 dark:text-
emerald-400" aria-hidden />
        Assigned Files
      </h3>

      <div className="space-y-3 h-[500px] overflow-y-auto">
        {assigned.length > 0 ? (
          assigned.map((file) => (
            <FileCard
              key={file.id}
              file={file}
              isAvailableSection={false}
            />
```

```
            ))
          ) : (
            <div className="text-center py-8 text-slate-500 dark:text-slate-400">
              {!clientId || !programId
                ? "Select a client and program from the sidebar to view assigned files"
                : "No assigned files match your search"}
            </div>
          )}
        </div>
      </div>
    </div>
  </div>
);
}
```

change update button color to green like assign which line do i need to change

Just **one place** needs to change ✅

Change this button inside `FileCard` menu

Find this block in your `uploadedfileslist.tsx`:

TypeScript

```
<button
  onClick={() => openUpdateModal(file)}
  className="w-full px-3 py-2 text-sm rounded bg-blue-600 text-white hover:bg-blue-700"
>
  Update
</button>
```

Replace with this

TypeScript

```
<button
  onClick={() => openUpdateModal(file)}
  className="w-full px-3 py-2 text-sm rounded bg-emerald-600 text-white hover:bg-emerald-700"
>
  Update
</button>
```

✅ Exact thing changed

Before:

TypeScript

```
bg-blue-600 hover:bg-blue-700
```

After:

TypeScript

```
bg-emerald-600 hover:bg-emerald-700
```

📍 Where it is in your file

Inside:

- `function FileCard(...)`
- inside `openMenu === file.id`
- inside the **Assigned section** (`isAvailableSection ? (...) : (...)`)
- the **Update** button

If you want, I can also give you **1 small UI improvement**:

👉 make **Update = green**, **Unassign = amber**, **Delete = red**, **Assign = green outline or solid**, so all actions look more consistent and professional.

Is this conversation helpful so far?